```
irb(main):005:0> fg 1
=> #<IRB::Irb: @context=#<IRB::Context:0x0000000106560960>,
   @signal_status=:IN_EVAL, @scanner=#<RubyLex:0x0000000106567258>>
irb#1(main):005:0> result
=> ["cat", "dog", "elk"]
irb#1(main):006:0>
```

In this example, we start a job and set result to the list of the first five Fibonacci numbers, then use the irb command internally to start a second job, setting result to something else, and then switch back and forth to show that the namespaces are independent.

Bindings

If you specify an object when you create a subsession, that object becomes the value of self in that binding. This is a convenient way to experiment with objects. In the following example, we create a subsession with the string "wombat" as the default object. Methods with no receiver will be executed by that object.

```
irb
irb(main):001:0> self
=> main
irb(main):002:0> irb "wombat"
irb#1(wombat):001:0> self
=> "wombat"
irb#1(wombat):002:0> upcase
=> "WOMBAT"
irb#1(wombat):003:0> size
=> 6
irb#1(wombat):004:0> gsub(/[aeiou]/, '*')
=> "w*mb*t"
irb#1(wombat):005:0> irb_exit
=> #<IRB::Irb: @context=#<IRB::Context:0x0000000108fa9e80>,
    @signal_status=:IN_EVAL, @scanner=#<RubyLex:0x00000001090be8e8>>
irb(main):003:0> self
=> main
irb(main):004:0> upcase
(irb):4:in `<main>': undefined local variable
  or method `upcase' for main:Object (NameError)
Did you mean?  case
  from /[elided]/gems/irb-1.4.1/exe/irb:11:in `<top (required)>'
```

Configuring irb

irb is remarkably configurable. You can set configuration options with command-line options, from within an initialization file, and while you're inside irb itself.

irb uses an initialization file in which you can set commonly used options or execute any required Ruby statements. When irb is run, it will try to load an initialization file from one of the following sources in order: ~/.irbrc, .irbrc, irb.rc, _irbrc, and $irbrc.